

FileEditSelectionViewGoRun3rd Yr Week 1 Module

EXPLORER

OPEN EDITORS

3RD YR WEEK 1 MODULE

star_pattern_square.java

star_pattern_square.java

star_pattern_square > main(String[])

1 public class star_pattern_square {
2 public static void main(String[] args) {
3 for (int i = 0; i < 5; i++) {
4 for (int j = 0; j < 5; j++) {
5 System.err.print(s: "*");
6 }
7 System.out.println();
8 }
9 }
10 }
11 }

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

GIT LENS

PS C:\Users\sachi\Desktop\3rd Yr Week 1 Module>

...
tar_pattern_square.java } ; if (\$?) { java star_pattern_square }

PS C:\Users\sachi\Desktop\3rd Yr Week 1 Module>

OUTLINE

TIMELINE

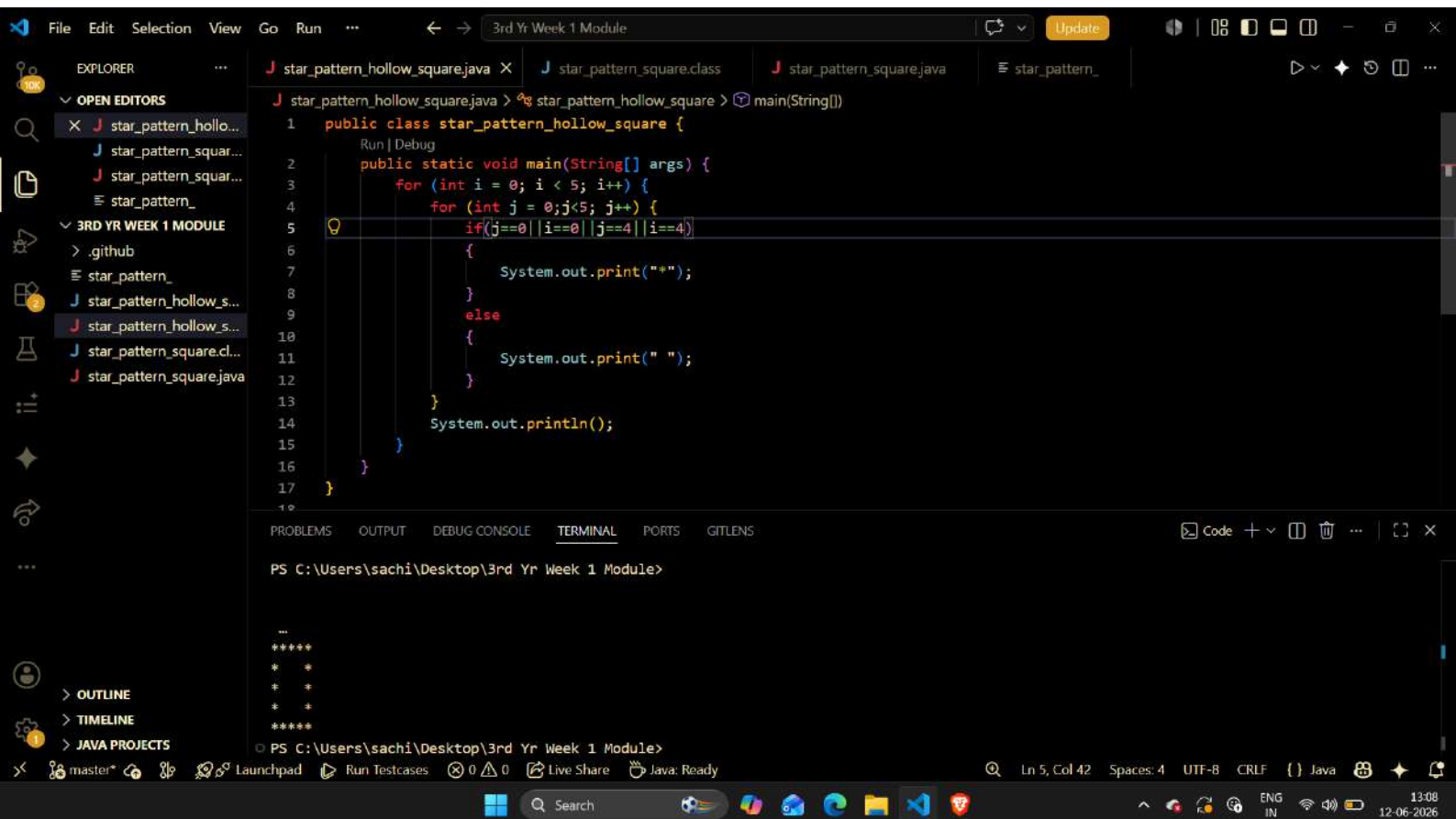
JAVA PROJECTS

master* Launchpad Run Testcases Live Share Java: Ready

Ln 4, Col 42 Spaces: 4 UTF-8 CRLF {} Java

11:22 12-06-2026

Tip: Try the Plan agent to research and plan before implementing changes.
+ J star_pattern_squan
Describe what to bu...
+ | | ...
| | | | Def...



File Edit Selection View Go Run ... 3rd Yr Week 1 Module Update

EXPLORER

OPEN EDITORS

- star_pattern_hollo...
- star_pattern_triang...
- star_pattern_squar...
- star_pattern_squar...
- star_pattern_...

3RD YR WEEK 1 MODULE

- .github
- star_pattern_...
- star_pattern_hollow_s...
- star_pattern_hollow_s...
- star_pattern_square.cl...
- star_pattern_square.java
- star_pattern_triangle.cl...
- star_pattern_triangle.j...

star_pattern_triangle.java > % star_pattern_triangle > main(String[])

```
1 public class star_pattern_triangle {
2     public static void main(String[] args) {
3         for (int i = 0; i < 5; i++) {
4             for (int j = 0; j < 5; j++) {
5                 if(j<=i)
6                 {
7                     System.out.print(s: "");
8                 }
9             }
10            System.out.println();
11        }
12    }
13 }
14
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\sachi\Desktop\3rd Yr Week 1 Module>

...
*
**

PS C:\Users\sachi\Desktop\3rd Yr Week 1 Module>

Ln 3, Col 38 Spaces: 4 UTF-8 CRLF {} Java

13:14 12-06-2026

File Edit Selection View Go Run ... 3rd Yr Week 1 Module Update

EXPLORER

OPEN EDITORS

- star_pattern_hollo...
- star_pattern_triang...
- star_pattern_invert...

3RD YR WEEK 1 MODULE

- .github
- star_pattern_
- star_pattern_hollow_s...
- star_pattern_hollow_s...
- star_pattern_inverted_...
- star_pattern_inverted_...
- star_pattern_square.cl...
- star_pattern_square.java
- star_pattern_triangle.d...
- star_pattern_triangle.j...

star_pattern_inverted_triangle.java

```
1 public class star_pattern_inverted_triangle {
2     public static void main(String[] args) {
3         for (int i = 0; i < 5; i++) {
4             for (int j = 0; j < 5; j++) {
5                 if (i <= j) {
6                     System.out.print(" ");
7                 }
8             }
9             System.out.println();
10        }
11    }
12 }
13
14
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\sachi\Desktop\3rd Yr Week 1 Module>

```
...
*****
****
***
**
*
```

PS C:\Users\sachi\Desktop\3rd Yr Week 1 Module>

Ln 6, Col 37 Spaces: 4 UTF-8 CRLF {} Java

13:22 12-06-2026

File Edit Selection View Go Run ... 3rd Yr Week 1 Module

Update

EXPLORER

OPEN EDITORS

- star_pattern_hollo...
- star_pattern_triang...
- star_pattern_invert...
- star_pattern_5.java

3RD YR WEEK 1 MODULE

- .github
- star_pattern_...
- star_pattern_5.class
- star_pattern_5.java
- star_pattern_hollow_s...
- star_pattern_hollow_s...
- star_pattern_inverted_...
- star_pattern_inverted_...
- star_pattern_square.cl...
- star_pattern_square.java
- star_pattern_triangle.cl...
- star_pattern_triangle.j...

star_pattern_hollow_square.java

star_pattern_triangle.java

star_pattern_inverted_triangle.java

star_pattern_5.java

```
1 public class star_pattern_5 {
2     public static void main(String[] args) {
3         int n=5;
4         for (int i = 0; i < n; i++) {
5             for (int j = 0; j < n; j++) {
6                 if(j<n-i-1)
7                     System.out.print(s: " ");
8             }
9             for (int j = 0; j < (2*i)+1; j++) {
10                 System.out.print(s: "*");
11             }
12             System.out.println();
13         }
14     }
15 }
16
17
18
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
cd "c:\Users\sachi\Desktop\3rd Yr Week 1 Module\" ; if ($?) { javac star_pattern_5.java } ;
if ($?) { java star_pattern_5 }
***
****
*****
*****
```

PS C:\Users\sachi\Desktop\3rd Yr Week 1 Module>

Ln 8, Col 43 Spaces: 4 UTF-8 CRLF {} Java

13:35 12-06-2026

File Edit Selection View Go Run Terminal Help 3rd Yr Week 1 Module Update

EXPLORER

- OPEN EDITORS
 - star_pattern_hollow...
 - star_pattern_triang...
 - star_pattern_invert...
 - star_pattern_5.java
 - star_pattern_6.java
- 3RD YR WEEK 1 MODULE
 - .github
 - star_pattern_...
 - star_pattern_5.class
 - star_pattern_5.java
 - star_pattern_6.class
 - star_pattern_6.java
 - star_pattern_hollow_s...
 - star_pattern_hollow_s...
 - star_pattern_inverted_...
 - star_pattern_inverted_...
 - star_pattern_square.cl...
 - star_pattern_square.java
 - star_pattern_triangle.cl...
 - star_pattern_triangle.j...
 - tempCodeRunner... 5

star_pattern_6.java

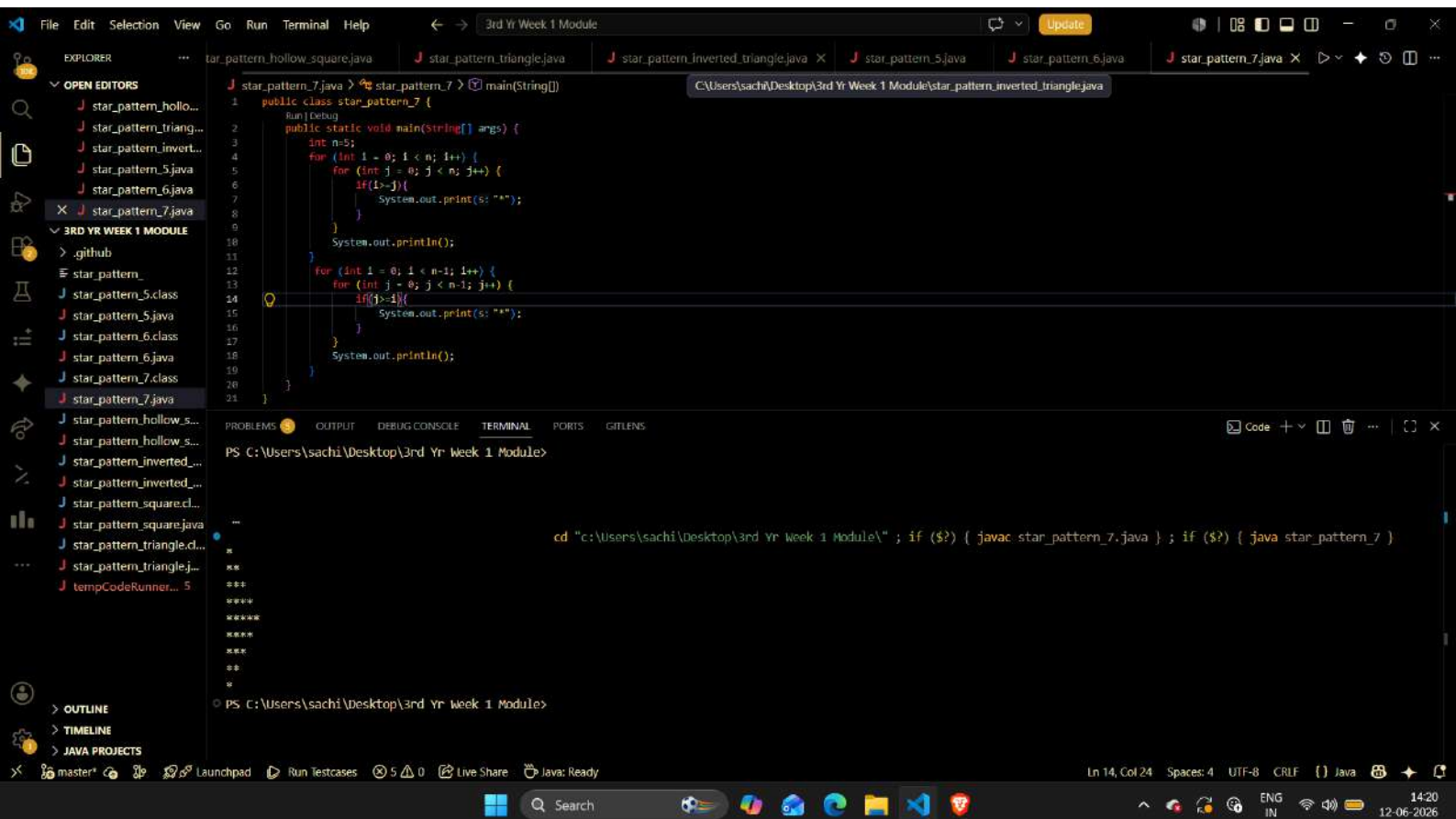
```
1 public class star_pattern_6 {
2     public static void main(String[] args) {
3         int n = 5;
4         for (int i = 0; i < n; i++) {
5             for (int j = 0; j < n; j++) {
6                 if (j < n - i - 1) {
7                     System.out.print(" ");
8                 }
9             }
10            for (int j = 0; j < (2 * i) + 1; j++) {
11                System.out.print("*");
12            }
13            System.out.println();
14        }
15        n=n-1;
16        for (int i = 0; i < n; i++) {
17            for (int j = 0; j < n; j++) {
18                if (i < j) {
19                    break;
20                }
21            }
22            System.out.print(" ");
23        }
24        for (int j = 0; j < (2*n)-(2*i)-1; j++) {
25            System.out.print("*");
26        }
27        System.out.println();
28    }
29 }
30
31 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GIT LENS

PS: C:\Users\sachi\Desktop\3rd Yr Week 1 Module>

master Launchpad Run Testcases 5 0 Live Share Java: Ready

Ln 21, Col 22 Spaces: 4 UTF-8 CRLF Java 14:13 12-06-2026



File Edit Selection View Go Run Terminal Help

3rd Yr Week 1 Module

Update

EXPLORER

OPEN EDITORS

- square.java number pattern
- increasing.java number pattern
- right_triangle.java number pattern
- inverted_diamond.java number p...
- sandwich_pattern.java number pa...
- increment_diamond.java number ...
- star_pattern_hollow_square.class
- Star_pattern_8.class
- Star_pattern_8.java star pattern

3RD YR WEEK 1 MODULE

- star pattern
 - star_pattern_...
 - star_pattern_5.class
 - star_pattern_5.java
 - star_pattern_6.class
 - star_pattern_6.java
 - star_pattern_7.class
 - star_pattern_7.java
 - Star_pattern_8.class
 - Star_pattern_8.java
 - star_pattern_hollow_square.class
 - star_pattern_hollow_square.java
 - star_pattern_inverted_triangle.class
 - star_pattern_inverted_triangle.java
 - star_pattern_square.class
 - star_pattern_square.java
 - star_pattern_triangle.class
 - star_pattern_triangle.java
 - tempCodeRunnerFile.java
- OUTLINE
- TIMELINE
- JAVA PROJECTS

star pattern > Star_pattern_8.java > Star_pattern_8 > main(String[])

```
1 public class Star_pattern_8 {
2     public static void main(String[] args) {
3         int n=5;
4         for (int i = 0; i < n; i++) {
5             for (int j = 0; j < n; j++) {
6                 if (i==j) {
7                     break;
8                 }
9                 else{
10                    System.out.print(" ");
11                }
12            }
13            for (int j = 0; j < n-i ; j++) {
14                System.out.print(" * ");
15            }
16            System.out.println();
17        }
18    }
19 }
20
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

Code - star pattern

PS C:\Users\sachi\Desktop\3rd Yr Week 1 Module\star pattern>

```
f ($?) { java Star_pattern_8 }
+ ~ ~ ~
* ~ ~
* ~
*

```

PS C:\Users\sachi\Desktop\3rd Yr Week 1 Module\star pattern>

```
cd "C:\Users\sachi\Desktop\3rd Yr Week 1 Module\star pattern\" ; if ($?) { javac Star_pattern_8.java } ; i
```

Ln 13, Col 44 Spaces: 4 UTF-8 CRLF {} Java

15:52 12-06-2026

File Edit Selection View Go Run Terminal Help 3rd Yr Week 1 Module Update

EXPLORER

OPEN EDITORS

star_pattern_5.class
star_pattern_5.java...
square.java numbe...

3RD YR WEEK 1 MODULE

.github
number pattern
square.class
square.java
star pattern

number pattern > square.java > square > main(String[])

```
1 public class square {  
2     Run | Debug  
3     public static void main(String[] args) {  
4         int n=5;  
5         for (int i = 0; i < n; i++) {  
6             for (int j = 0; j < n; j++) {  
7                 System.out.print(s: "1");  
8             }  
9             System.out.println();  
10        }  
11    }  
12 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

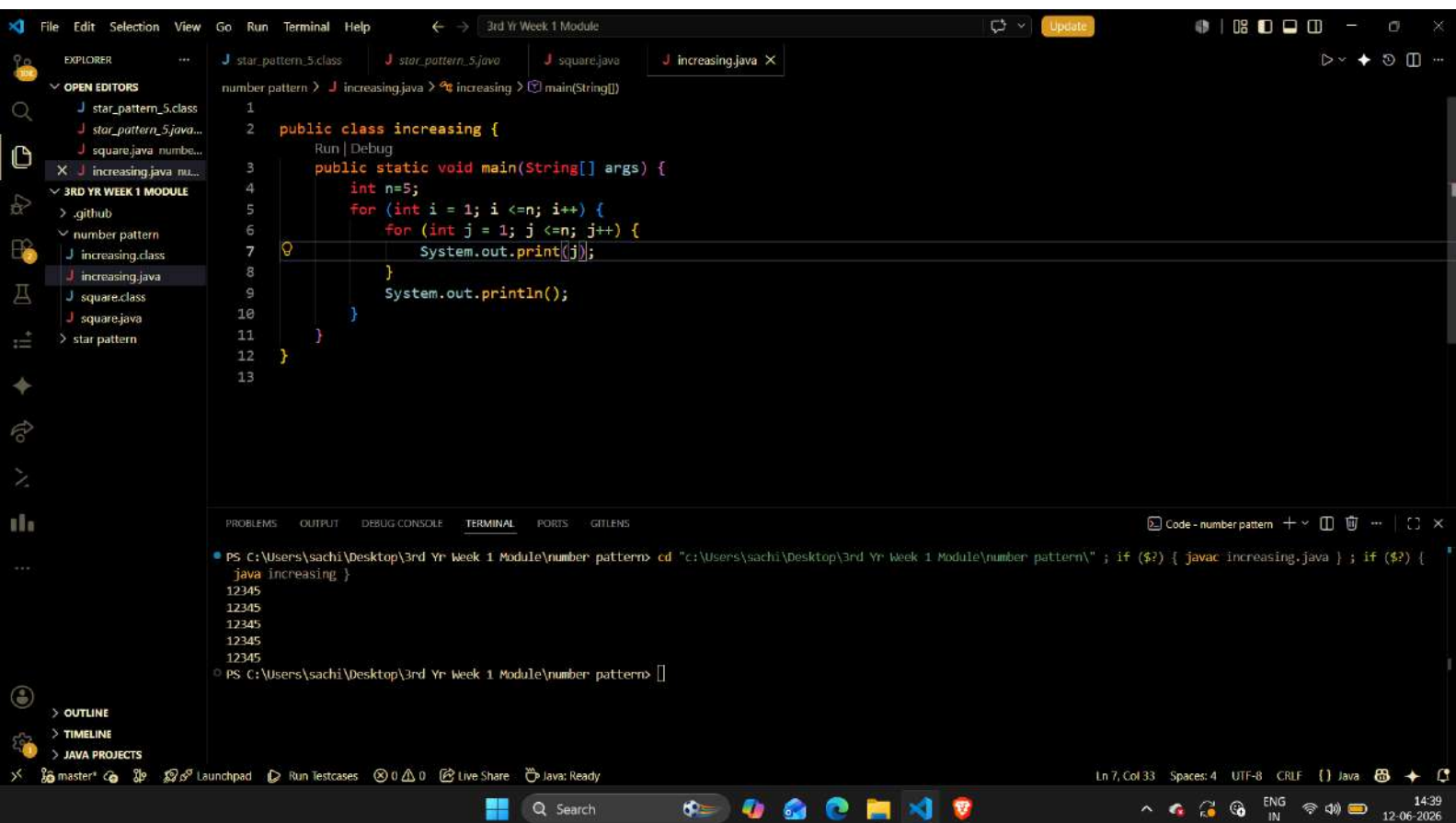
Code - number pattern

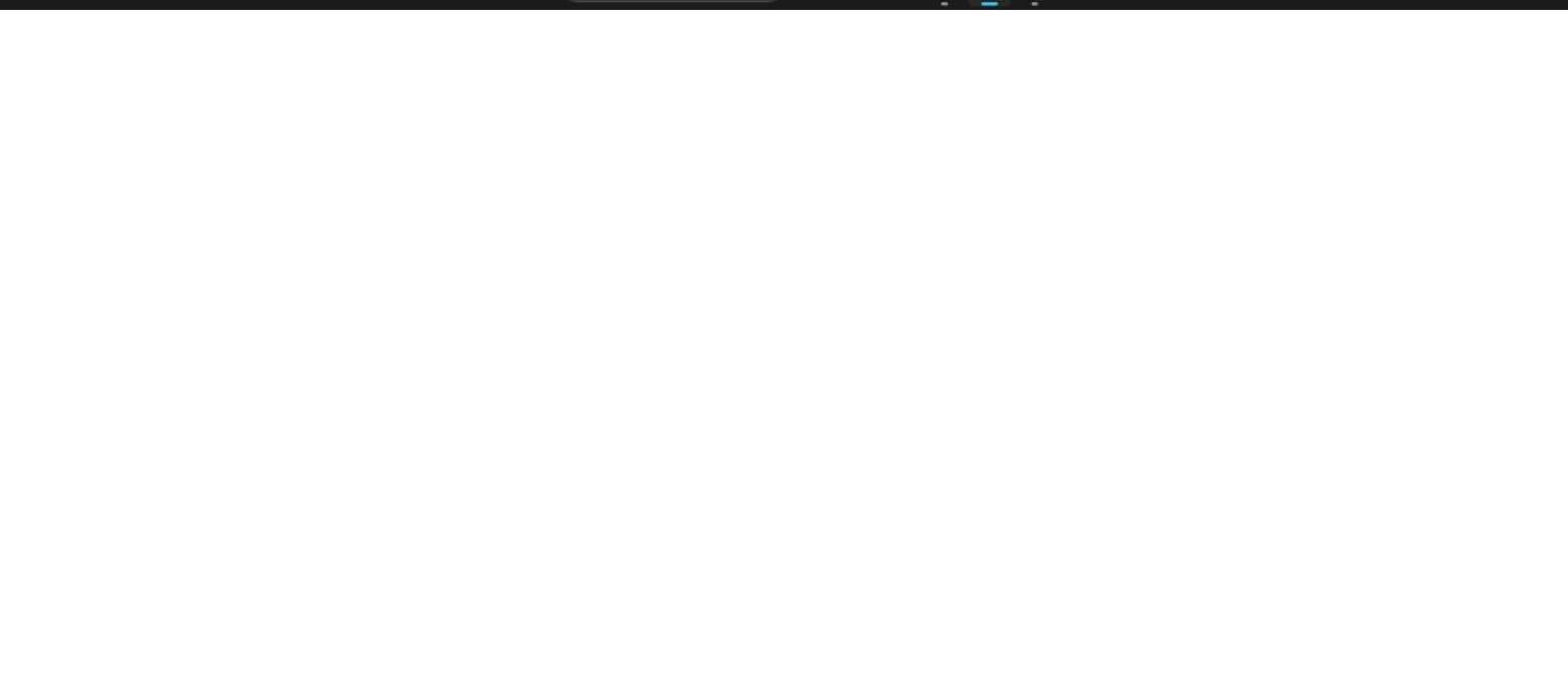
```
cd "c:\Users\sachi\Desktop\3rd Yr Week 1 Module\number pattern\" ; if ($?) { javac square.java } ; if ($?) { java  
square }  
11111  
11111  
11111  
11111  
11111  
11111
```

PS: C:\Users\sachi\Desktop\3rd Yr Week 1 Module\number pattern>

Ln 6, Col 29 Spaces: 4 UTF-8 CRLF {} Java

14:37 12-06-2026





File Edit Selection View Go Run Terminal Help 3rd Yr Week 1 Module Update

EXPLORER

- star_pattern_5.class
- square.java
- increasing.java
- right_triangle.java
- inverted_diamond.java
- Star_pattern_8.java

3RD YR WEEK 1 MODULE

- .github
- number pattern
 - increasing.class
 - increasing.java
 - inverted_diamond.class
 - inverted_diamond.java
 - right_triangle.class
 - right_triangle.java
 - square.class
 - square.java
 - star pattern

number pattern > inverted_diamond.java > inverted_diamond > main(String[])

```
2 public class inverted_diamond {
3     public static void main(String[] args) {
4         int n=5;
5         for (int i = 0; i < n; i++) {
6             for (int j = 0; j < n; j++) {
7                 if (i==j)
8                     break;
9             }
10            else{
11                System.out.print(s: " ");
12            }
13        }
14        for (int j = 0; j < n-i ; j++) {
15            System.out.print(i+3 + " ");
16        }
17        System.out.println();
18    }
19 }
20 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

Code - number pattern

PS C:\Users\sachi\Desktop\3rd Yr Week 1 Module\number pattern>

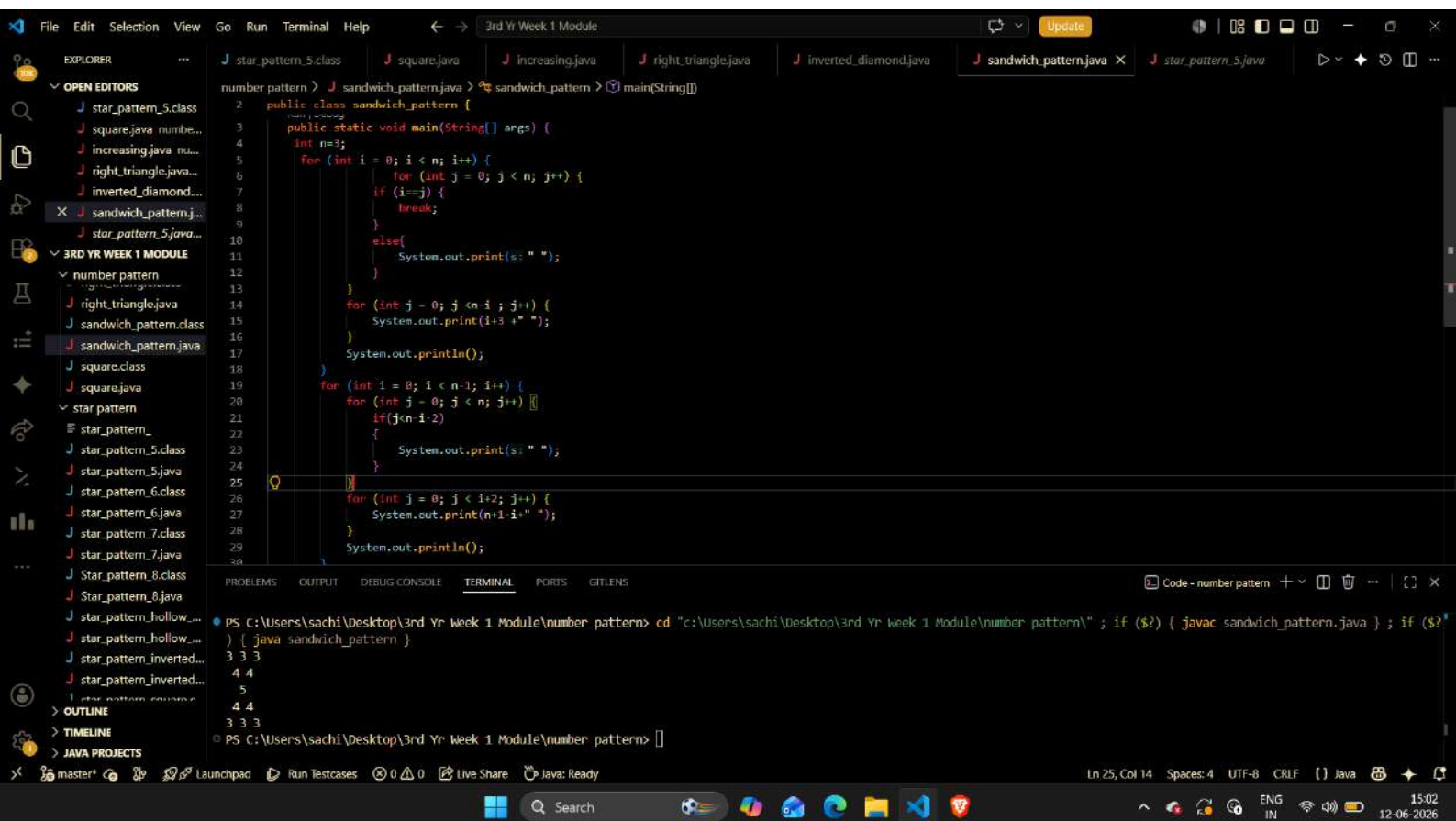
```
cd "C:\Users\sachi\Desktop\3rd Yr Week 1 Module\number pattern" ; if ($?) { javac inverted_diamond.java } ; if ($?) { java inverted_diamond }
```

3 3 3 3 3
4 4 4 4
5 5 5
6 6
7

PS C:\Users\sachi\Desktop\3rd Yr Week 1 Module\number pattern>

Ln 9, Col 18 Spaces: 4 UTF-8 CRLF {} Java

14:45 12-06-2026



File Edit Selection View Go Run Terminal Help 3rd Yr Week 1 Module Update

EXPLORER

- OPEN EDITORS
 - star_pattern_5.class
 - square.java numbe...
 - increasing.java nu...
 - right_triangle.java...
 - inverted_diamond...
 - sandwich_pattern.j...
 - increment_diamond...
 - inverted_diamond...
- 3RD YR WEEK 1 MODULE
 - number pattern
 - increasing.class
 - increasing.java
 - increment_diamond...
 - increment_diamondj...
 - inverted_diamond.cl...
 - inverted_diamond.java
 - right_triangle.class
 - right_triangle.java
 - sandwich_pattern.class
 - sandwich_pattern.java
 - square.class
 - square.java
 - star pattern
 - star_pattern_...
 - star_pattern_5.class
 - star_pattern_5.java
 - star_pattern_6.class
 - star_pattern_6.java
 - star_pattern_7.class
 - OUTLINE
 - TIMELINE
 - JAVA PROJECTS

number pattern > increment_diamond.java > increment_diamond > main(String[])

```
1
2 public class increment_diamond {
3     public static void main(String[] args) {
4         int n=5;
5         int count=0;
6         for (int i = 0; i < n; i++) {
7             for (int j = 0; j < n; j++) {
8                 if(i>=j){
9                     System.out.print(i+3);
10                    count=i+3;
11                }
12            }
13            System.out.println();
14        }
15        for (int i = 0; i < n-1; i++) {
16            for (int j = 0; j < n-1; j++) {
17                if(j>=i){
18                    System.out.print(count-i-1);
19                }
20            }
21            System.out.println();
22        }
23    }
24 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\sachi\Desktop\3rd Yr Week 1 Module\number pattern>

Ln 18, Col 47 Spaces: 4 UTF-8 CRLF [] Java 15:07 12-06-2026



Pattern - 1 (Square Pattern)

Pattern :

```

* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
  
```

Logic :

- ① Use outer loop (i) for rows.
- ② Use inner loop (j) for columns.
- ③ For every row, print 5 stars using the inner loop.
- ④ After printing 5 stars, move to the next line.

Outer Loop → Number of rows (5)

Inner Loop → Number of stars in each row (5)

Dry Run :

```

i = 0 → j = 0, 1, 2, 3, 4 → * * * * * println()
i = 1 → j = 0, 1, 2, 3, 4 → * * * * * println()
i = 2 → j = 0, 1, 2, 3, 4 → * * * * * println()
i = 3 → j = 0, 1, 2, 3, 4 → * * * * * println()
i = 4 → j = 0, 1, 2, 3, 4 → * * * * * println()
  
```

Code :

```

public class star_pattern_square {
    public static void main(String[] args) {
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 5; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
  
```

One-Line Formula to Remember :

```

Square Pattern
Rows = Outer Loop
Columns = Inner Loop
for every row
print fixed number of stars
  
```



Pattern - 2 (Hollow Square Pattern)

Pattern :

```

* * * * *
*           *
*           *
*           *
*           *
* * * * *
  
```

Logic :

- ① Use outer loop (i) for rows.
- ② Use inner loop (j) for columns.
- ③ Print "*" when we are on boundary.
 - $i == 0 \rightarrow$ first row (top boundary)
 - $i == 4 \rightarrow$ last row (bottom boundary)
 - $j == 0 \rightarrow$ first column (left boundary)
 - $j == 4 \rightarrow$ last column (right boundary)
- ④ Otherwise print space inside the square.

Boundary (Top, Bottom, Left, Right)	$\rightarrow$ Print "*"
Inside (Center part)	$\rightarrow$ Print space

Dry Run :

```

i = 0  →  j = 0, 1, 2, 3, 4  →  * * * * *  (top boundary)
i = 1  →  j = 0 * j = 1, 2, 3 space j = 4 * →  *      *
i = 2  →  j = 0 * j = 1, 2, 3 space j = 4 * →  *      *
i = 3  →  j = 0 * j = 1, 2, 3 space j = 4 * →  *      *
i = 4  →  j = 0, 1, 2, 3, 4  →  * * * * *  (bottom boundary)
  
```

Code :

```

public class star_pattern_hollow_square {
    public static void main(String[] args) {
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 5; j++) {
                if (j == 0 || i == 0 || j == 4 || i == 4) {
                    System.out.print("*");
                }
                else {
                    System.out.print(" ");
                }
            }
            System.out.println();
        }
    }
}
  
```

One-Line Formula to Remember :

if ($j == 0 \parallel i == 0 \parallel j == 4 \parallel i == 4$)	$\rightarrow$ Print "*"
else	$\rightarrow$ Print space

Pattern - 3 (Right Triangle Pattern)

Pattern :

```

*
* *
* * *
* * * *
* * * * *

```

Logic :

- ① Use outer loop (i) for rows.
- ② Use inner loop (j) for columns.
- ③ Print "*" when $j \leq i$
- ④ As row number increases, number of stars increases.

Row 0	→	1 star
Row 1	→	2 stars
Row 2	→	3 stars
Row 3	→	4 stars
Row 4	→	5 stars

Dry Run :

```

i = 0 → j = 0 → *
i = 1 → j = 0, 1 → * *
i = 2 → j = 0, 1, 2 → * * *
i = 3 → j = 0, 1, 2, 3 → * * * *
i = 4 → j = 0, 1, 2, 3, 4 → * * * * *

```

Code :

```

public class star_pattern_triangle {
    public static void main(String[] args) {
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 5; j++) {
                if (j <= i) {
                    System.out.print("*");
                }
            }
            System.out.println();
        }
    }
}

```

One-Line Formula to Remember :

$j \leq i \rightarrow$ Print "*"

Shortcut Memory Trick :

Stars increase row by row.

Row Number + 1 = Number of Stars



Pattern - 1 (Inverted Triangle Pattern)

Pattern :

```
* * * * *
* * * *
* * *
* *
*
```

Logic :

- ① Use outer loop (i) for rows (0-4).
- ② Use inner loop (j) for columns (0-4).
- ③ Print "*" when $j \geq i$ (for left-aligned triangle).
- ④ As row number increases, number of stars decreases.

Row 0	→ Prints 5 stars
Row 1	→ Prints 4 stars
Row 2	→ Prints 3 stars
Row 3	→ Prints 2 stars
Row 4	→ Prints 1 star

Dry Run :

```
i = 0 → j = 0,1,2,3,4 → prints * * * * *
i = 1 → j = 0(n/a), j = 1,2,3,4 → prints * * * *
```

Code :

```
public class star_pattern_inverted_triangle {
    public static void main(String[] args) {
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 5; j++) {
                if (j >= i) {
                    System.out.print("*");
                } else {
                    System.out.print(" ");
                }
            }
            System.out.println();
        }
    }
}
```

One-Line Formula to Remember :

$j \geq i \rightarrow$ Print "*"

Shortcut Memory Trick :

Stars start aligned and decrease row by row.
Columns printed are from row index onwards.

Pattern - 5 (Pyramid Pattern)

Pattern :

```

      *
     ***
    *****
   ********
  *********
 
```

Logic :

- ① Use outer loop (i) for rows (0-4).
- ② For each row, first print spaces using loop (j).
- ③ Then print stars using a second loop (j) (Stars:'). The number of stars is $2i+1$.
- ④ Go to the next line.

Row 0	→ Spaces: 4, Stars: 1
Row 1	→ Spaces: 3, Stars: 3
Row 2	→ Spaces: 2, Stars: 5
Row 3	→ Spaces: 1, Stars: 7
Row 4	→ Spaces: 0, Stars: 9

Dry Run

i = 0	→ spaces: j = 0,1,2,3	→ prints " "
	stars: j = 0	→ prints "*" "
i = 1	→ spaces: j = 0,1,2	→ prints " " "
	stars: j = 0,1,2	→ prints "***" "

Code :

```

public class star_pattern_5 {
    public static void main(String[] args) {
        int n=5;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n-i-1; j++) {
                System.out.print(" ");
            }
            for (int j = 0; j < (2*i)+1; j++) {
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}

```

One-Line Formula to Remember :

spaces = $n-i-1$, stars = $2i+1$

Shortcut Memory Trick :

Spaces decrease, Stars increase with odd numbers ($2i+1$). Symmetry on center star.

Pattern - 6 (Diamond Pattern)

Pattern :

```

      *
     ***
    *****
   *****
  *****
 *
  
```

Logic :

- ① Divide diamond into upper & lower halves.
- ② Upper half (Rows 0-4): Pyramid logic (see Pattern-5).
- ③ Lower half (Rows 5-8): Inverted Pyramid logic.
- ④ Go to next line after each row.

Upper Half: Spaces = $n-i-1$, Stars = $2i+1$.
Lower Half ($n=4$ for this half): Spaces = $i-1$, Stars = $2(n-i+1)-1$.

Dry Run (Upper & Lower) :

```

i = 0 (Upper): spaces: j = 0,1,2,3 → prints " "
                stars:  j = 0       → prints "***"
i = 1 (Lower): spaces: j = 0       → prints " "
                stars:  j = 0,1,2,3,4 → prints "*****"
  
```

Code :

```

public class star_pattern_6 {
    public static void main(String[] args) {
        int n=5;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n-i-1; j++) {
                System.out.print(" ");
            }
            for (int j = 0; i <= n; i++) {
                for (int j = 0; j < 2*(n-i)+1; j++) {
                    System.out.print("*");
                }
            }
        }
    }
}
  
```

One-Line Formula to Remember :

spaces = $(n-i-1)$ & stars = $(2i+1)$ for Upper,
spaces = i & stars = $2(n-i)+1$ for Lower.

Shortcut Memory Trick :

Combine Pyramid & Inverted Pyramid logic. Key is to handle lower loop limits correctly. Symmetry on center.

Pattern - 7 (Left Half Diamond / Right Triangle Comb.)

Pattern :

```

      *
     * * *
    * * * * *
   * * * * *
  * * *
 * *

```

Logic :

- ① Divide diamond into upper & lower halves.
- ② Upper half (Rows 0-4): Standard right triangle pattern.
(i for rows, j for stars, j for $i \leq i$).
- ③ Lower half (Rows 5-8): Inverted Right Triangle. Wait... the dry run uses j for stars, $j < n-1$, print * line when $j \geq n-i-1$ (inverted columns).
- ④ Go to next line.

Upper Half: $i=0$ to 4, print $(i+1)$ stars
 Lower Half: $i=0$ to 3, print spaces and then decreasing stars (from 4 to 1) [Formula for spaces: $n-i-1$, formula for stars: for $n-i$]

Dry Run :

$i=0$ (Upper): $j=0 \rightarrow$ prints *
 $i=4$ (Upper): $j=0,1,2,3,4 \rightarrow$ prints *****
 $i=0$ (Lower): spaces: $j=0,1,2,3 \rightarrow$ prints " ", stars: $j=0$
 $i=3$ (Lower): spaces: $j=0 \rightarrow$ stars: $j=0 \rightarrow$ prints *

```

// Lower Half
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n; j++) {
        if (j >= n - i - 1) {
            System.out.print("*");
        } else {
            System.out.print(" ");
        }
    }
    System.out.println();
}

```

Code :

```

public class star_pattern_7 {
    public static void main(String[] args) {
        int n=5;
        // Upper Half
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (j <= i) {

```

One-Line Formula to Remember :

$(j \leq i)$ Upper, $(j \geq n-i-1)$ Lower $\rightarrow$ Print "*"

Shortcut Memory Trick :

Upper is normal, Lower is inverted spaces and stars. Symmetry is on rows 4 & 5.

Pattern : Pattern - 8 (Combination Pattern Breakdown)

Logic :

- ① Upper Half (Rows 0-4): Symmetrical Inverted Triangle. Logic: leading spaces ($j < i$),
Logic: leading spaces ($j < i$), star loop boundary ($2*(n-i-1)+1$)
- ② Combination Core (Diamond Part): Identify intersection.
- ③ Upper row (space-star separation): Use leading spaces for Leading spaces ($j < i-1$), followed any space-break logic if ($i==j$) break; inside a full boundary loop ($j < n-1$)
- ④ Lower part: normal symmetrical diamond. Go to next line.

Combo Core Breakdown:
Row i boundary: leader=i, break logic for middle-space (if $i==j$ break).
Boundary logic preserves space.
E.g., Row 1 leader 1, Row 2 leader 2.

Dry Run :

i = 0 (Upper) → spaces: j=0 → prints " "; breaking loop (no check); prints *****
i = 1 (Upper) → spaces: j=0 → prints " " leader; breaking loop (breaks at j=1); prints *****
i = 4 (Combination): spaces: j=0,1,2,3 leader → prints " " leader; breaking loop (breaks at 4); prints diamond base

Code :

```
public class Star_pattern_8 {
    public static void main (String[] args) {
        int n=5;
        for (int i = 0; i < n; i++) {
            // Space break logic for Combo Part (with Correction to full width n-1)
            for (int j = 0; j < n; j++) {
                if (i==j) // Red break highlight
                    else { System.out.print(" ");
            }
            // for (int j = 0; j < n-i j++) for full width n-i...
            System.out.print("**");
        }
        System.out.println();
    }
}
```

→ Correction to break logic

One-Line Formula to Remember :

Formula (Upper): row i space leader = i, stars = n-i.
Symmetrical boundary needed.

Shortcut Memory Trick :

Upper half always uses space leader equal to row number i. Stars are n/2 determines combo crossover.

Pattern : Number Pattern - 1 (Square with specific digits)

```
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
```

Logic :

- ① Use outer loop (i) for rows (0-4).
- ② For each row, use inner loop (j) for columns (0-4).
- ③ In both loops, simply print the digit '1' and a space.
- ④ Go to the next line.

Row 0 → Prints '1 1 1 1 1'
Row 1 → Prints '1 1 1 1 1'
Row 2 → Prints '1 1 1 1 1'
Row 3 → Prints '1 1 1 1 1'
Row 4 → Prints '1 1 1 1 1'

Dry Run :

i = 0 → prints " " leader (0 spaces); then loop j=0,1,2,3,4 prints '1 1 1 1 1'
i = 1 → prints " " leader (0 spaces); then loop j=0,1,2,3,4 prints '1 1 1 1 1'

Code :

```
public class square {
    public static void main (String[] args) {
        int n=5;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print("1 "); // Print "1 " literally
            }
            System.out.println();
        }
    }
}
```

One-Line Formula to Remember :

Print literal "1 ".

Shortcut Memory Trick :

Nested loops for rows/columns.
Inner print is fixed character "1 "
N determines NxN size.

Pattern :Number Pattern - 2 (Increasing Digits Square)

```

1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5

```

Logic :

- ① Use outer loop (i) for rows (1-5).
- ② For each row, use inner loop (j) for digits (1-5).
- ③ In the inner loop, print the current digit 'j' and a space.
- ④ Go to the next line.

Row 1	→ Prints	'1 2 3 4 5'
Row 2	→ Prints	'1 2 3 4 5'
Row 3	→ Prints	'1 2 3 4 5'
Row 4	→ Prints	'1 2 3 4 5'
Row 5	→ Prints	'1 2 3 4 5'

Dry Run :

i = 1 → prints " " leader; then loop j=1,2,3,4,5 prints '1 2 3 4 5'

i = 2 → prints " " leader; then loop j=1,2,3,4,5 prints '1 2 3 4 5'

Code : `public class increasing {`
 `public static void main (String[] args) {`
 `int n=5;`
 `for (int i = 1; i <= n; i++) {`
 `for (int j = 1; j <= n; j++) {`
 `System.out.print(j + " ");` // Print digit 'j' and space
 `System.out.println();`
 `}`
 `}`
`}`

One-Line Formula to Remember :Print Variable "j"Shortcut Memory Trick :

Nested loops (1 to N inclusive).
 Outer is row, Inner is digit sequence.
 The digit variable 'j' is printed.

Pattern - X (Consecutive Rows Triangle)Pattern :

```

      1
     2 2
    3 3 3
   4 4 4 4
  5 5 5 5 5

```

Logic :

- ① Use inner loop variable boundary for alternating rows.
- ② Odd rows ($i=0,2,4,\dots$) $\rightarrow$ print full boundary row width, Boundary boundary = 1 for leading spaces ($j < n-i-1$).
- ③ Even rows ($i=1,3$) $\rightarrow$ print full boundary row width, Boundary boundary = 1 for leading spaces ($j < n-i-1$), boundary loop boundary changes.
- ④ Go to next line. (Preserving previous structure).

Combo Core Breakdown :
Boundary changes boundary per row.
 Odd rows $i=0,2,4$ boundary = 1.
 Even rows $i=1,3$ boundary = 1.
 Leading space logic boundary $n-i-1$ preserves boundary.

Dry Run :

- $i=0$ (Odd row) $\rightarrow$ leader space $j=0,1,2,3$ boundary $\rightarrow$ prints _ _ _ _; inner boundary=1
 $i=1$ (Even row) $\rightarrow$ leader space $j=0,1,2$ boundary $\rightarrow$ prints _ _ _; inner boundary=1
 $i=4$ (Odd row) $\rightarrow$ leader space (none); inner boundary change (boundary=1); print 5.

Code :

```

public class Star_pattern_X {
    public static void main(String[] args) {
        int n=5;
        for (int i = 0; i < n; i++) {
            // Space logic
            for (int j = 0; j < n-i-1; j++) {
                System.out.print(" ");
            }
            // Corrected consecutive row boundary logic
            int currentBoundary = 1; // Correction
            for (int j = 0; j < 2*i+1; j++) {
                System.out.print(i+1); // Correction to print actual row value i+1
            }
            System.out.println();
        }
    }
}

```

 $\rightarrow$ Correction to logicOne-Line Formula to Remember :

Formula: spaces ($j < n-i-1$), then consecutive row values ($i+1$) with a boundary.

Shortcut Memory Trick :

Odd rows 0,2,4 boundary=1, Even rows 1,3 boundary=1. Start values 1-5 inclusive. $N/2$ crossover determines alternating.

Number Pattern - 6 (Inverted Left-Half Diamond)Pattern :

```

- - - 3 3 3 3 3
- - - 4 4 4 4
- - 5 5 5
- 6 6
7

```

Logic :

- ① Use inner loop variable boundary (n-i) for decreasing row width.
- ② Leading space logic: Row i has n-i-1 leading spaces to make a Left-Half shape. (e.g., j < n-i-1)
- ③ Decreasing values: Value changes and resets per row. Use variable int value = 3+i; (now, j ges-1). Value decreases with loop.
- ④ Go to next line. (Preserving the previous structure).

Core Breakdown :

Row width = n-i.
 Row start value changes:
 Boundary = 1 for rows (j < n-i-1).
 Loop starts at value, boundary
 Starts at row i=0 start value=3.
 Value resets to 3+i for next row.

Dry Run :

i=0 → leading space j=0,1,2,3 → prints ___; inner boundary change currentVal = 3
 i=1 → leading space j=0,1,2 → prints __; inner boundary change prints 4 4 4 4
 i=4 → leading space (none); inner boundary change currentVal = 7; prints 7.

Code :

```

public class Star_pattern_6 {
    public static void main (String[] args) {
        int n=5;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < i; j++) {
                System.out.print(" ");
            }
            // Corrected Left-Half Inverted Diamond logic
            int boundary = n - i;
            int currentBoundaryValue = 3 + i; // Correction to width
            for (int j = 0; j < currentBoundaryValue; j++) { // Correction to starting value
                System.out.print(currentBoundaryValue);
            }
            System.out.println();
        }
    }
}

```

→ Correction to logic

One-Line Formula to Remember :

Formula: spaces (j < i), then (n-i) values of (3+i).

Shortcut Memory Trick :

Starts rows 1-5 inclusive. Crossover determines N/2 halves. Odd row boundary= (3+i) start value. Left-half shape n-i-1 spaces. Width decreases n-i.

Number Pattern - 7 (Combined Sandwich Pattern: Symmetrical-Aligned Combo)

Pattern :

```

      - 3 3 3 -
      - 4 4 -
      - 4 4 -
      - 3 3 3 -
  
```

Logic :

- ① Use inner loop variable boundary for alternating rows.
- ② Odd rows ($i=0,2,4,\dots$) $\rightarrow$ print full boundary row width, **Boundary** boundary = 1 for leading spaces ($j < n-i-1$).
- ③ Even rows ($i=1,3$) $\rightarrow$ print full boundary row width, **Boundary** boundary = 1 for leading spaces ($j < n-i-1$), boundary loop boundary changes.
- ④ Go to next line. (Preserving previous structure).

Combined Pattern Crossover Breakdown :
Rows $i=0,2,4$ boundary = 1.
Rows $i=1,3$ boundary = 1.
Leading space $n-i-1$ preserves boundary.
Value calculation: Top values $3+i$
Bottom values $n+1-i$ (to reverse)

Dry Run :

```

i=0 Top: spaces  $\rightarrow j=0,1,2$  prints " " "; prints 3 3 3 (Value  $3+0$ )
i=1 Top: spaces  $\rightarrow j=0,1$  prints " " "; prints 4 4 (Value  $3+1$ )
i=0 Bottom: spaces  $\rightarrow j=0,1$  prints " " "; prints 4 4 (Value  $5-1-0=4$ )
i=1 Bottom: spaces  $\rightarrow j=0,1,2$  prints " " "; prints 3 3 3 (Value  $5-1-1=3$ )
  
```

Code : public static void main (String[] args) { (Fix dry run values and space counts to match new shape)

```

    int n=5;
    for (int i = 0; i < n; i++) {
        // Space logic
        for (int j = 0; j < n-i-1; j++) {
            System.out.print(" ");
        }
        // Corrected consecutive logic
        int currentBoundary = 1;
        for (int j = 0; j < 2*i+1; j++) {
            System.out.print(i + 1);
        }
        System.out.println();
    }
  
```

Correction: Enforce full symmetrical centering:
For leading space, j must go to $(n-i-1)$ in both top and bottom sandwich parts, not $n-i-2$, to prevent shifting.
This ensures a clean center line.

One-Line Formula to Remember :

Formula: spaces $(n-i-1)$, values $3+i$ (Top) / $n+1-i$ (Bottom). N determines alignment.

Shortcut Memory Trick :

$n+1-i$ (Bottom). N determines alignment. Symmetry point determines logic crossover.

Number Pattern - 8 (Incrementing Half-Diamond Combo: Symmetrical-Aligned Combo)

Pattern :



Logic :

- ① Use inner loop variable boundary for alternating rows.
- ② Top-right triangle (Rows 0-4) → prints decreasing number **Boundary** bvints by row value (i+3) with no space separator.
- ③ Bottom-inverted triangle (Rows 5-8) → prints increasing number of spaces (j < n+i-9), for row value (j < n+i-9), followed by row value (count-i+4).
- ④ Go to next line. (Preserving previous structure).

Combined Pattern Crossover Breakdown :
Rows i=0,2,4,5,7,9 boundary=1
Rows i=1,3,6,8 boundary = 1.
Leading space n-i-1 preserves boundary.
Value calculation: Top values i+3
Bottom values count-i+4 (to reverse values)

Dry Run :

- i=0 Top: spaces → j=0,1,2,3 prints " " leader; breaking loop (breaks at j=4); (Value 0+3)
- i=1 Top: spaces → j=0,1,2 prints " " leader; breaking loop (breaks at j=3); (Value 1+3)
- i=5 Top/Bottom Crossover: spaces (none); prints 77777 (Value 1+3)
- i=6 Bottom: spaces → j=0 prints " " leader; prints 6666 (Value 7-6+4=5)

Code :

```
public static void main (String[] args) {
    public static void main (String[] args) {
        int n=5;
        for (int i = 0; i < n; i++) {
            // Correction
            int (j>=n-i) (> if count -i+1)
            System.out.print (count - i + 1);
        }
        System.out.println();
    }
}
```

Correction to value variable logic:
Print count - i - 1 for bottom part
value decrement.

Correction to break logic

One-Line Formula to Remember :

Formula: spaces (n-i-1), values i+3 (Top) / count-i+1 (Bottom), values i+3 (Top) / determines halves.

Shortcut Memory Trick :

n+1-1 (Bottom). N determines count-i+1 (Bottom). Boundary n determines halves.